Individual Contribution

ResourceX — Asset & Resource Allocation Management System

Name	Yaswanth Kumar	**Roll No**	**S20240010264** (Member 4)
Project	ResourceX — Asset & Resource Allocation Management System	**Branch**	`feature/staff-operations-portal`
Assigned Scope	React implementation of the **Staff Operations Portal**, specifically focusing on the four assigned operational modules: **Allocations**, **Inventory**, **Maintenance**, and **Returns**, alongside staff portal navigation, components, and backend API integration.		
Total Impact	2 primary commits, 19 files developed or modified, **+4,108 lines added**, 414 lines removed.		

1. Summary of My Work

As **Member 4** of the ResourceX engineering team, I was assigned full ownership of the **Staff Operations Portal**. In the original static HTML prototype, staff actions were handled via direct DOM manipulation with static mock data. I developed a complete, responsive, and robust React implementation of the Staff Operations Portal connected directly to the live NestJS backend API.

My work specifically covers four core operational modules:

- **Allocations (Allocation Requests View):** Built the end-to-end resource assignment flow. Approved requests from department members are matched against available inventory resources of the requested category. Implemented a robust multi-select dropdown enabling staff to assign exact quantities of inventory items, validate selections against request quotas, execute allocation calls to `/api/requests/:id/allocate`, and filter requests by status (Pending / Allocated / All) and search keywords.
- **Inventory (Department Resource Management):** Created a comprehensive inventory auditing and management view. Staff can browse department assets, perform live full-text search across serial numbers and resource types, filter by status (Available, In Use, Maintenance, etc.), trigger the controlled **Add Resource Modal** (which dynamically populates available resource types from `/api/resources/catalog`), edit existing resource specifications via **Edit Resource Modal**, execute scrap actions, and view attached procurement invoice documents.
- **Maintenance (Repairs & Asset Health):** Developed the maintenance management interface tracking items requiring physical attention. Staff can inspect requests in `Maintenance Requested` status, formally accept tickets to transition items to `Maintenance`, record repair completion to restore assets to `Available` with condition updated to `Good`, scrap irreparably damaged equipment, and review expandable historical repair logs.
- **Returns (Check-in & Physical Inspection):** Engineered the return processing pipeline for returned assets. Staff inspect items returned by requestors (status `Returned`), evaluate physical wear to record inspection condition (`Good`, `Average`, `Bad`), process returns to re-enter inventory or route damaged assets to maintenance, and audit the complete historical log of return operations.
- **Portal Architecture & Supporting Infrastructure:** Developed the master `StaffDashboard.jsx` routing container with URL synchronisation, universal sidebar integration, toast notifications (`StaffToast.jsx`), invoice file previews (`FilePreviewModal.jsx`), and the dedicated client service layer `src/services/staff.js`.

2. Commits

Commit	Message	Part	Size
`9e5cf24`	feat(staff): implement complete staff operations portal matching specification and staff.html	React Frontend & Backend seed support	19 files, +3936 / -94
`0d6c38b`	fix(staff): prevent multi-select dropdown clipping and align allocations table with staff.html	React Frontend (UI alignment & dropdown styling)	3 files, +172 / -320

3. Files I Developed or Modified

New = *File created from scratch by me.* *Modified* = *Existing codebase file updated to integrate staff operations.*

File & Path	Change	Functional Responsibility & What It Does
`AllocationsView.jsx` react-frontend/src/components/dashboards/staff/views/	New	**Allocations Module (Module 1):** Renders allocation request queue; filters requests by department and status; fuzzy-matches approved request categories against available stock; handles multi-select resource selection with counter badges; executes batch allocation dispatch via `/api/requests/:id/allocate`.
`InventoryView.jsx` react-frontend/src/components/dashboards/staff/views/	New	**Inventory Module (Module 2):** Master catalogue table of department assets; provides search by serial/type/vendor; status filtering; modal triggers for adding and editing resources; scrap action dispatch; supports document preview for linked invoice files.
`AddResourceModal.jsx` react-frontend/src/components/dashboards/staff/components/	New	**Inventory Component:** Modal form to register new assets; dynamically fetches department catalogue types; validates model, serial number, vendor, condition, and optional procurement invoice link.
`EditResourceModal.jsx` react-frontend/src/components/dashboards/staff/components/	New	**Inventory Component:** Modal form allowing staff to edit serial numbers, location, condition, and operational notes of existing inventory records.
`FilePreviewModal.jsx` react-frontend/src/components/dashboards/staff/components/	New	**Inventory Component:** In-app viewer modal rendering PDF and image attachments linked to procurement invoices for verifiable audit compliance.
`MaintenanceView.jsx` react-frontend/src/components/dashboards/staff/views/	New	**Maintenance Module (Module 3):** Manages active repair queue; transitions items from `Maintenance Requested` to `Maintenance`; marks repaired assets back to `Available` with `Good` condition; scraps broken assets; renders expandable historical repair log.
`ReturnsView.jsx` react-frontend/src/components/dashboards/staff/views/	New	**Returns Module (Module 4):** Manages asset returns queue; enables physical condition assessment (Good, Average, Bad); processes return check-ins via `/api/returns/:id/process`; displays expandable return history audit table.
`StaffDashboard.jsx` react-frontend/src/components/dashboards/staff/	New	**Staff Shell & Router Hub:** Implements navigation tab switching across all staff sub-views; synchronises active view with URL routes; manages global toast notification state; coordinates modal preview states.
`StaffDashboard.css` react-frontend/src/components/dashboards/staff/	New	**Styling Architecture:** Complete dedicated CSS rules for staff tables, badges, filter chips, action buttons, multi-select dropdown positioning, and modal overlays.

File & Path	Change	Functional Responsibility & What It Does
`StaffStatsHeader.jsx` react-frontend/src/components/dashboards/staff/components/	New	**Metrics Component:** Renders live summary stat cards (Pending Allocations, Total Resources, Active Maintenance, Pending Returns) with department scoping.
`StaffToast.jsx` react-frontend/src/components/dashboards/staff/components/	New	**Feedback Component:** Auto-dismissing toast notifications informing staff of successful allocations, updates, and validation errors.
`staff.js` react-frontend/src/services/	New	**API Service Layer:** Comprehensive client API module interfacing with `/api/requests`, `/api/resources`, `/api/maintenance`, `/api/returns`, `/api/procurements`, and `fetchStaffDashboardStats`.
`DashboardView.jsx` react-frontend/src/components/dashboards/staff/views/	New	**Supporting View:** Executive dashboard overview aggregating urgent action cards and department operational metrics for quick staff triage.
`RegistrationView.jsx` react-frontend/src/components/dashboards/staff/views/	New	**Supporting View:** Dedicated barcode and batch resource registration interface for incoming stock dispatches.
`ProcurementTasksView.jsx` react-frontend/src/components/dashboards/staff/views/	New	**Supporting View:** Staff view to log procurement purchases, vendor details, and register delivered assets.
`WarehouseOverview.jsx` react-frontend/src/components/dashboards/staff/views/	New	**Supporting View:** Warehouse storage zone and rack layout visualization for department inventory tracking.
`data.service.ts` backend/src/data/	Modified	Added backend data service methods to support staff allocation lookups and department scoping consistency.
`seed.ts` backend/src/data/	Modified	Updated initial seed records to ensure valid mock resources and request types match staff department categories for immediate testing.

4. React Concepts Demonstrated

- **Component Architecture, Props & Callback Communication:** Clean separation of concerns between top-level controller views and reusable visual components. `StaffDashboard` coordinates active views and toasts; `AllocationsView`, `InventoryView`, `MaintenanceView`, and `ReturnsView` delegate user interactions to dedicated modal components (`AddResourceModal`, `EditResourceModal`, `FilePreviewModal`) via props and callbacks (`onConfirm`, `onClose`, `addToast`).
- **State Management & Event Handling:** Extensive usage of React hooks (`useState`, `useCallback`, `useMemo`, `useEffect`). State tracks real-time data queues, search queries, active status filters, selected items in multi-select dropdowns, loading indicators, and active modal overlays. Event handlers process user clicks, input changes, and form submissions with inline validation.
- **Controlled Forms:** Built robust controlled components for resource creation, editing, condition evaluation, and return inspection. Input fields (model, serial number, vendor, condition dropdowns) maintain synchronised local state with immediate visual error feedback.
- **Side Effects & Asynchronous API Lifecycles:** Used `useEffect` combined with memoized `useCallback` loaders to fetch asynchronous data from backend REST endpoints upon component mounting. Automatic state refresh triggers after mutations (allocation completion, resource addition, maintenance acceptance, return processing) ensure real-time UI synchronisation.
- **Authentication & Context Integration:** Integrated `useAuth()` context to securely extract the logged-in staff member's department, enforcing organizational and departmental scoping across all queries and operations.

5. How the Staff Operations Portal is Structured

Layer	Component / File	Responsibilities
Router & Shell	`StaffDashboard.jsx`	Synchronises tab state with URL path, mounts universal sidebar and header, manages toast container, and handles invoice preview modals.
Styling	`StaffDashboard.css`	Provides responsive layout styling, card grids, badge colors, table alignment, and dropdown positioning.

Core Module Views	`AllocationsView.jsx`
	`InventoryView.jsx`
	`MaintenanceView.jsx`
	`ReturnsView.jsx`
		Module 1: Matching and allocating approved requests.
		Module 2: Inventory stock auditing, search, filter, scrap.
		Module 3: Maintenance queue, accepting repair, marking repaired.
		Module 4: Returns inspection, condition grading, check-in.

Modals & Widgets	`AddResourceModal.jsx`
	`EditResourceModal.jsx`
	`FilePreviewModal.jsx`
	`StaffStatsHeader.jsx`
	`StaffToast.jsx`
		Provides isolated, reusable user interface overlays for asset creation, metadata editing, invoice attachment verification, department metrics summary, and operation feedback toasts.

Layer	Component / File	Responsibilities
API Client	`services/staff.js`	Centralized asynchronous HTTP client communicating with backend endpoints via JWT authentication.

6. Testing and Verification Done

- **Vite Build Compilation Check:** Executed `npm run build` in `react-frontend/`; compiles cleanly with zero syntax, JSX, or bundling errors.
- **Backend API Validation:** Tested all staff API endpoints against live NestJS server (port 3001) verifying correct status codes and JSON payloads:
  - `GET /api/requests?department=IT Services` → 200 OK (returns department requests).
  - `GET /api/resources?department=IT Services` → 200 OK (returns inventory catalogue).
  - `GET /api/maintenance/history` → 200 OK (returns historical repair logs).
  - `GET /api/returns/history` → 200 OK (returns historical return records).
  - `POST /api/requests/:id/allocate` → Validated successful allocation of selected resource IDs.
- **Manual End-to-End Walkthrough:**
  - **Allocations:** Logged in as Staff (`yaswanth@resourcex.com` / `12345678`), navigated to *Allocation Requests*, expanded multi-select dropdown on pending request, selected matching available inventory item, clicked *Allocate* → Verified request moved to *Allocated* and toast notification appeared.
  - **Inventory:** Filtered inventory by status, searched by serial number, opened *Add Resource* modal, verified catalogue dropdown populates correctly, and opened *Edit Resource* modal.
  - **Maintenance:** Inspected repair queue, accepted ticket to transition item to `Maintenance`, and verified repair action returned asset to `Available` with condition `Good`.
  - **Returns:** Inspected return queue, selected inspected condition (`Good`), clicked *Process Return* → Verified asset returned to active available stock.